

NABL Document Intelligence

Full Project Report

Status as of 2026-09-02. This report covers the whole project: the live 3-phase applicant assistant (backend/ + frontend/) and the offline dataset-labeling-to-benchmark pipeline documented in the companion report (*Lab_Report_Pipeline_Report.pdf*). Every count below was read from the real source tree or a real output file, not estimated.

Repo: NABL-Document-Intelligence (backend/ FastAPI + SQLite, frontend/ React+Vite+TS)

Companion report: docs/Lab_Report_Pipeline_Report.pdf (dataset pipeline detail)

1. Overview of this document

NABL Document Intelligence is an assistant for NABL lab accreditation and recognition-scheme applications, covering all 9 real NABL application forms (151, 152, 153, 153A, 154, 155, 157, 158, 159). It has three phases — an eligibility wizard, a document ingestion pipeline (OCR + LLM classification + LLM extraction), and a human-in-the-loop (HITL) review UI — built on a FastAPI + SQLite backend and a React/Vite/TypeScript frontend.

A separate, later initiative reuses this same OCR/LLM machinery to build and benchmark a general lab-report extraction pipeline (medical/food/milk/chemical test reports, not NABL application forms) — that work has its own companion report and is only summarized here in Section 7's design discussion.

2. Goal

Let an applicant lab complete NABL paperwork mostly by uploading documents it already has (certificates, CVs, PT/ILC reports, or even a filled copy of the form itself) instead of retyping everything by hand — while gating that upload behind a conversational check that the lab actually meets the prerequisites, and giving a human a clear, source-grounded review step before anything is treated as final.

3. Requirements

3a. Functional

- Conversational eligibility wizard before upload unlocks: 4 ISO-readiness questions for the 6 full-accreditation forms, a lighter 2-question gate for the 3 recognition-scheme forms — a deterministic state machine decides pass/fail, the LLM only parses the free-text reply into a verdict.
- Multi-file upload (PDF / DOCX / JPG / PNG) per application.
- Route each upload by its real shape: born-digital PDF → PyMuPDF text + true bounding boxes; DOCX → python-docx text (no pixel boxes); scanned page in devanagari/arabic/ta/te/ka → the deployed deeppreef-ocr AWS Lambda; scanned English page → local RapidOCR in-process, with a vision-LLM call as a last-resort fallback.
- Classify every upload into one of 17 supporting-document types (GST/CIN proof, equipment cert, staff CV, PT/ILC result, ...), or as a full `completed_application_form` if it's a filled copy of the form itself.
- Extract fields into the correct Pydantic schema for the application's chosen form (9 forms, 2 schema bases — full accreditation vs. lightweight recognition).
- Deterministic regex extraction for GST/PAN/TAN numbers ahead of any LLM call, since those have an unambiguous, checkable format.
- For the whole-filled-form case specifically: page-level chunking, embedding retrieval per schema section, per-section LLM extraction, then a targeted verification retry pass for fields that came back missing.
- Merge every document's fields into one compiled form object, detecting and reporting cross-source conflicts rather than silently picking one value.
- HITL split-pane review UI: source document with highlighted region on the left, the editable pre-filled form on the right, low-confidence fields flagged, a chat drawer for questions or re-extraction.
- Render the compiled form back out to a filled PDF.

3b. Non-functional

- No paid API key required — the LLM chain tries each configured provider in order and falls through on any failure, returning a 503 naming every failure only if all of them fail.
- A failed document extraction is marked status: "failed" with a clear error; it never crashes the request.
- Every LLM/OCR call is mocked in the test suite — the full suite runs with no API key and no live AWS/Ollama/Qdrant service.
- Reuse the real sibling deepreef-ocr engine in-process for English (no vendored copy, no extra network hop).
- Re-extracting or uploading a second document never mixes its retrieved chunks into another document's results (one vector-store collection per document).

4. Background

The 9-form schema (`backend/app/schemas/forms.py/common.py`) is modeled directly on the official NABL templates, read in full rather than sampled: 151/152/153/153A/154/158 are full ISO/IEC 17025 or ISO 15189 accreditation applications sharing one schema base, while 155/157/159 are lighter PT-performance-based recognition schemes sharing a second, narrower base. NABL 100B is a pure accreditation-procedure document with no applicant fields and isn't modeled as a form. The whole-form (`completed_application_form`) extraction path needs a wide context window — a real filled form runs 15–30 pages (100K+ characters), which in practice only Gemini's free-tier context budget comfortably covers.

5. Who's your customer

- **Primary:** a NABL applicant lab's staff, filling out one of 9 accreditation/recognition forms and uploading the certificates/records they already have.
- **Secondary:** the internal engineering team, who need the extraction pipeline's failure modes to be visible (per-field confidence, conflicts, a structured extraction report) rather than hidden inside a single opaque "success".
- **Downstream:** whoever at the lab (or a NABL assessor) reviews the compiled, rendered PDF before it's actually submitted.

6. Tenets

- Deterministic before LLM wherever the format allows it — the wizard's gating logic and GST/PAN/TAN extraction are both rule-based, not model calls.
- Never crash on a bad document or an unlucky provider outage — degrade to a clear per-document or per-request error instead.
- Free-tier friendly — the product must be fully runnable without a paid API key.
- Every extracted field should be source-grounded (traceable to a page/region), not just a bare value with no provenance.
- Reuse existing infrastructure (the LLM chain, the deepreef-ocr engine) across every new pipeline rather than growing a second copy.

7. Design

7a. Challenge

Nine real forms split into two genuinely different shapes, so one flat schema can't represent all of them. Supporting documents vary from a one-page GST certificate to a 30-page filled form, and from clean born-digital PDFs to photographed pages in five different non-English scripts. A whole filled-in form needs a fundamentally different extraction strategy than a single-purpose certificate — sending 100K+ characters to one LLM call isn't reliable across free-tier providers. And free LLM/OCR providers are each individually unreliable (rate limits, small context windows, intermittent outages), so no single provider can be a hard dependency.

7b. Design options considered

Decision	Options	Chosen
OCR routing	(A) one OCR engine for everything (B) route by script: Lambda for 5 non-English scripts, local RapidOCR for English, vision-LLM as last resort	B
Whole-form extraction	(A) one giant LLM call over the full text (B) page-chunk + embed + retrieve top-k per schema section + targeted verification retry for missing fields	B
LLM provider strategy	(A) one fixed provider (B) ordered multi-provider fallback chain, tried in sequence per call	B

7c. Pros

- **Script-routed OCR:** English (the common case) never needs AWS or a network call at all; non-English scripts still get a purpose-built model instead of a generic OCR engine's worse-than-nothing guess.
- **Chunk+retrieve+verify:** only the chunks relevant to a schema section are sent per extraction call instead of the whole document every time; the verification retry recovers fields the first pass missed without re-asking about everything.
- **Multi-provider chain:** the product works with zero paid spend, and one provider's rate limit or outage doesn't take the feature down.

7d. Cons

- **Script-routed OCR:** the vision-LLM last-resort fallback has no reliable per-field bounding box — the review UI can only show a page-level highlight for those fields, not a tight box (see README's "Known limitation").
- **Chunk+retrieve+verify:** only wired up for the whole-form path; every other doc_type still sends its (much shorter) text directly, so there are two different extraction code paths to maintain, not one.
- **Multi-provider chain:** free-tier model quality and JSON-mode/vision support vary by provider (only Gemini supports vision here), so which provider actually answered a given call is not guaranteed, and whole-form extraction in practice only works reliably when Gemini specifically is available.

System map (real modules, backend/app/)

Phase	Module(s)	Role
Wizard	wizard/	Deterministic prerequisite gate; LLM only parses free-text answers.
Ingestion	documents/pipeline.py, classifier.py, extractor.py	Route by file type, classify doc_type, extract fields.

Phase	Module(s)	Role
Ingestion (whole-form)	documents/chunking.py, embeddings.py, vector_store.py, retrieval.py, verification.py	Page-chunk → embed → Qdrant → retrieve top-k → per-section extract → verify missing fields.
Ingestion (deterministic)	documents/rule_extraction.py	Regex GST/PAN/TAN extraction ahead of any LLM call.
OCR	documents/local_ocr.py, ocr_client.py	Local RapidOCR (English) / deepreef-ocr Lambda (5 scripts).
Compile	documents/compiler.py, extraction_report.py	Merge fields into the form shape; detect conflicts; report what's missing/rejected.
Review (HITL)	routers/review.py, documents/form_pdf.py	Serve the compiled form + extraction report; field-patch endpoint; render filled PDF.
LLM chain	llm/factory.py, chain.py, providers.py	Ordered multi-provider fallback (10 services, 12 configured chain entries incl. key rotation).
API	routers/wizard.py, documents.py, review.py, chat.py	12 REST endpoints (Section 10a).
Frontend	frontend/src/pages, components	WizardPage, UploadPage, ReviewPage; DocumentViewer, FormField, ChatPanel.

8. Conclusion

The product is code-complete across all three phases and genuinely tested: 258 tests pass across 28 test files (3,982 lines) with 0 failing, covering the wizard state machine, every LLM-chain/provider path, both OCR paths, the classifier, compiler and conflict detection, chunking/retrieval/verification, form-PDF rendering, and the full offline dataset pipeline — every external call (LLM, OCR, AWS, Qdrant) is mocked, so this suite runs with zero live services or API keys.

- 9 real NABL forms modeled across 2 schema bases; 17 supporting-document types classified automatically, plus the whole-filled-form path.
- 12 REST endpoints across 4 routers (wizard, documents, review, chat); a 3-page React frontend wired to all of them.
- LLM chain actually wires 10 distinct backends (Gemini, Groq, HuggingFace, Cerebras, OpenRouter, SambaNova, Mistral, Nvidia, GitHub, local Ollama) plus 3 key-rotation duplicates — more than the 3 providers (Gemini/Groq/HuggingFace) the current README documents; the README is stale on this specific point.
- Known, honestly-documented gaps (from the README, unchanged by this report): the vision-LLM OCR fallback has no per-field bounding box; NABL 154's Mobile-Laboratory/Complaints tables and 158's Affiliations/Family-members tables are schema-only, not wired to any doc_type yet; whole-form extraction is reliable mainly when Gemini specifically is reachable, given its context-size requirement.

This system and the dataset-labeling-to-benchmark pipeline (companion report) share their OCR and LLM-chain foundations but are otherwise independent efforts on different data (NABL application paperwork vs. general lab test reports) — the benchmark's 0.832 F1 result does not directly certify this application's field-extraction accuracy, though both draw on the same underlying extraction machinery and the same caveat applies to each: no step in either system has human-verified ground truth yet.

9. FAQs

Q: What happens if no LLM API key is configured at all?

A wizard/chat request returns 503 with a clear message. A document upload still completes, but the document is marked status: "failed" with a clear error rather than crashing the request.

Q: Does every extracted field come with a bounding box on the source document?

Born-digital PDF and local-OCR (English) fields do. DOCX fields don't (no rasterized page to box). The vision-LLM fallback path (only reached if RapidOCR itself fails, or for a script that's neither English nor one of deeppreef-ocr's five supported scripts) shows a page-level highlight only, not a tight box — a real, currently-open limitation.

Q: Is this the same pipeline as the benchmark report's 0.832 F1 result?

No. That benchmark scores a separate, later effort against real lab test reports (medical/food/milk/chemical), reusing this project's OCR and LLM-chain code but a different schema and a different dataset. See the companion report for that pipeline's own detail and caveats.

Q: Can the whole product run without any AWS account?

Yes, for every form and doc_type except scanned pages in devanagari/arabic/ta/te/ka specifically — those route to a deployed AWS Lambda. English scans, born-digital PDFs, and DOCX files never touch AWS.

Q: Why do 154 and 158 have schema fields that are never auto-filled?

Fidelity to the real NABL templates: 154's Mobile Laboratory/Complaints/FSSAI-scope/Sampling-scope tables and 158's Affiliations/Family-members/Related-labs/Contracts tables are modeled so the compiled JSON and review UI can hold and edit them, but no document type has been wired to populate them automatically yet — they're a manual-entry gap, not a bug.

10. Appendix

10a. Metrics

Area	Count
Backend test files / total lines / passing	28 files / 3,982 lines / 258 passed, 0 failed
Backend app modules (incl. dataset pipeline)	15 packages
documents/ module: files / lines	20 files / 1,882 lines
REST endpoints	12, across wizard.py / documents.py / review.py / chat.py
Frontend source files / lines	9 files (3 pages, 3 components, api client) / 882 lines
NABL forms modeled	9 (151, 152, 153, 153A, 154, 155, 157, 158, 159) across 2 schema bases
Supporting-document types classified	17, plus the whole-form (completed_application_form) path
LLM backends wired in llm/factory.py	10 distinct services + 3 key-rotation duplicates = 12 chain entries
DB tables (models.py)	4 (Application, Document, ExtractedField, ChatMessage)

REST endpoints

Method	Path	Router
POST	/applications	wizard
GET	/applications/{id}/wizard	wizard
POST	/applications/{id}/wizard/answer	wizard
POST	/applications/{id}/documents	documents
GET	/applications/{id}/documents	documents
GET	/documents/{id}/render	documents
POST	/documents/{id}/reextract	documents
GET	/applications/{id}/form	review
GET	/applications/{id}/extraction-report	review
GET	/applications/{id}/form.pdf	review
PATCH	/documents/{id}/fields/{field_id}	review
GET / POST	/applications/{id}/chat	chat

10b. Performance

Aspect	Note
Per-request LLM/OCR timing	Not centrally logged by the app; per-provider timeout is configurable (llm_timeout_seconds, default 30s).
Ollama (local) timeout	210s configured default — substantially longer than every hosted provider's 30s, consistent with local CPU inference being the slowest path in the chain.
Retrieval	all-MiniLM-L6-v2 embeddings (~90MB, 384-dim), top_k=4 chunks retrieved per schema section, on-disk Qdrant (no server process).
Confidence threshold	0.85 default — the line the review UI uses to flag a field as low-confidence rather than auto-accept it.

10c. Edge cases handled

- **Every configured LLM provider fails on one call** (e.g. all free tiers rate-limited at once) — 503 naming each provider's specific failure, not a silent 500.
- **A PDF is mostly digital but has a few scanned pages mixed in** (a signed declaration, a scanned annexure) — checked and OCR'd per page, not per document.
- **An uploaded document doesn't match any known doc_type** — classified as other rather than forced into the wrong bucket.
- **A whole filled-in form runs past what non-Gemini providers can accept** (100K+ characters) — fails with a clear “every provider failed” error instead of a truncated, silently-wrong extraction.
- **Two documents disagree on the same field's value** — recorded as a conflict in the extraction report, not silently overwritten.
- **Concurrent uploads hit the local vector store at the same time** — a single shared, lock-guarded Qdrant client, since local-mode Qdrant holds a file lock on its storage directory.

10d. Glossary

Term	Meaning
doc_type	One of 17 categories a supporting document is classified into (e.g. equipment_calibration_certificate), or completed_application_form for a filled copy of the form itself.
FormCategory	ACCREDITATION (151/152/153/153A/154/158, full ISO application) or RECOGNITION (155/157/159, lighter PT-performance scheme).
Grounding	Tying an extracted field's value back to the specific page/region of the source document it came from.
Chunk	One page-level unit of a document's text, used only by the whole-form retrieval path.
Retrieval top_k	How many chunks are fetched per schema section for the LLM to read, instead of sending the whole document.
Rule extraction	Deterministic regex extraction (GST/PAN/TAN) run before any LLM call, since those formats are unambiguous and checkable.
HITL	Human-in-the-loop — the review step where a person confirms or corrects auto-filled, especially low-confidence, fields.
Confidence threshold	The score below which a field is flagged for manual review instead of auto-accepted (default 0.85).

Term	Meaning
deepreef-ocr Lambda vs. local RapidOCR	The same underlying OCR engine; the Lambda serves 5 non-English scripts remotely, local RapidOCR serves English in-process.